DO NOW — RECALL

Binary Search — From Trace Table to Working Code

Name: _____ Class: _____ Date: _____

1. What index identifies the first element of a C# array?

2. How does a linear search differ from a binary search?

3. Given int[] numbers = {3, 7, 12, 18, 24}; what is numbers[3]?

4. Why can binary search miss a target that exists in an unsorted array?

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

0 — C# arrays are zero indexed.

2. Model answer

Linear search checks sequentially; binary search repeatedly halves the remaining sorted range.

3. Model answer

18 — the fourth item has index 3.

4. Model answer

Because comparisons rely on sorted order to discard one half safely.

By the end, you can…

• explain how binary search finds a value in a sorted array

• trace low, high and mid through the search

• implement, test and debug a binary search in C#

Vocabulary — know these terms

Recall the term — sorted array

Recall the term — target

Recall the term — midpoint

Recall the term — lower bound

Recall the term — upper bound

Recall the term — iteration

Recall the term — time complexity

NEW LEARNING

Read, annotate and explain

Find 31 in sorted data

Index is below each value. Yellow = midpoint; green = target.

Binary search requires sorted input. Cold call: If the middle value is too small, which indexes can you reject, and why?

Start with valid bounds

The last index is 7, not 8. Array length is not a valid index.

Find the midpoint

Integer division gives index 3. Compare its value (18) with target (31).

Discard half

All positions through index 3 are too small. Keep only 4–7.

Stop when found or exhausted

If low becomes greater than high, return -1. Never assume the target exists.

Explain one step in your own words, using the correct technical terms.

I DO — WORKED EXAMPLE

Follow the worked method

Write a function that returns the target's index

Declare an integer array

```
int[] numbers = {3, 7, 12, 18, 24, 31, 45, 60};
```

int = integer; [] = an array; numbers = its name; { } = the values; ; = end of the statement. The first index is 0.

Choose a target

```
int target = 31;
```

int declares an integer variable. The value 31 is what we want to find. This is not its index.

Declare a function that returns a position

```
static int BinarySearch(int[] data, int target)
{
```

The return type int means the function gives back a whole number: the position of the target or -1. int[] data receives the sorted array; int target receives the number to find.

Set the search bounds

```
int low = 0;
    int high = data.Length - 1;
```

low is the first valid index (0); high is the last valid index (7). Length is 8, but index 8 does not exist.

Repeat and find the middle

```
while (low <= high)
    {
        int mid = low + (high - low) / 2;
```

while repeats while a candidate remains; <= also checks when low and high are equal. Integer division gives a whole-number index.

Compare and move the bounds

```
if (data[mid] == target)
            return mid;
        else if (data[mid] < target)
            low = mid + 1;
        else
            high = mid - 1;
```

== checks equality; < means move right; otherwise move left. Each unsuccessful step removes half of the remaining candidates.

Finish the method

```
}
    return -1;
}
```

} closes the loop and the method. Returning -1 means the value was not found.

Call the method and print the result

```
int position = BinarySearch(numbers, target);
Console.WriteLine(position); // 5
```

Call the function from Main after declaring the array and target. Its returned integer is stored in position and printed. Here 31 is at index 5; if the target is absent, position contains -1.

WE DO — GUIDED PRACTICE

Try → Check → Improve

TRY in BLACK PEN. Complete the guided tasks before the Exit Ticket. CHECK the original source model later; IMPROVE errors in GREEN PEN.

Guided 1 — TRY: Trace the search for 13

Task: Trace BinarySearch(data, 13) . At each iteration record low , high , mid and data[mid] . Return the index, or -1 if absent.

Method / working:

Guided 2 — TRY: Trace the search for 13

Task: Trace BinarySearch(data, 13) . At each iteration record low , high , mid and data[mid] . Return the index, or -1 if absent.

Method / working:

Guided 3 — TRY: Trace the search for 13

Task: Trace BinarySearch(data, 13) . At each iteration record low , high , mid and data[mid] . Return the index, or -1 if absent.

Method / working:

Guided 4 — TRY: Trace the search for 13

Task: Trace BinarySearch(data, 13) . At each iteration record low , high , mid and data[mid] . Return the index, or -1 if absent.

Method / working:

Guided 5 — TRY: Trace the search for 13

Task: Trace BinarySearch(data, 13) . At each iteration record low , high , mid and data[mid] . Return the index, or -1 if absent.

Method / working:

Guided 6 — TRY: Trace the search for 13

Task: Trace BinarySearch(data, 13) . At each iteration record low , high , mid and data[mid] . Return the index, or -1 if absent.

Method / working:

Guided 7 — TRY: Trace the search for 13

WE DO — GUIDED PRACTICE

Try → Check → Improve · continued

Task: Trace BinarySearch(data, 13) . At each iteration record low , high , mid and data[mid] . Return the index, or -1 if absent.

Method / working:

Guided 8 — TRY: Trace the search for 13

Task: Trace BinarySearch(data, 13) . At each iteration record low , high , mid and data[mid] . Return the index, or -1 if absent.

Method / working:

Guided 9 — TRY: Complete the C# scaffold

Supply missing C# syntax: array length, while condition, midpoint and updating low.

```
int high = __________;
while (__________)
{
    int mid = __________;
    if (data[mid] < target)
    {
        __________;
    }
}
```

Method / working:

Guided 10 — TRY: Debug an incorrect branch

The midpoint value is smaller than the target. Identify the mistake in the code and correct it.

```
else if (data[mid] < target)
{
    high = mid - 1;
}
```

Method / working:

Guided 11 — TRY: Hinge question — ready to code?

Sorted array {2,5,8,12,17,21,30}, target 21. First midpoint is index 3, value 12. What happens to the bounds?

```
A: low=0, high=2
B: low=4, high=6
C: low=3, high=6
D: low=0, high=6
```

Method / working:

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations or a justified explanation.

Question 1

Before you write
Discuss with your partner: What two inputs must BinarySearch receive? What integer must it return when the target is found or missing? Why must the array be sorted?
Then: finish your discussion, face forwards and work silently and independently.

Question 2

Write the complete function
Write a C# function named BinarySearch that accepts a sorted integer array and a target integer.
Return the target's array index , or -1 if absent.
CODE:

```
int[] data = {4, 9, 13, 19, 26, 35, 42, 57};
int target = 35;
// Expected return value: 5
```

Write the function itself on your mini-whiteboard. Do not open .NET Fiddle yet.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations or a justified explanation.

Question 3

Code and test in .NET Fiddle
Once your teacher has checked your whiteboard, type your function into .NET Fiddle or your school IDE. Call it from Main , run the program and check the returned position.
CODE:
BinarySearch(data, 35); // 5
BinarySearch(data, 4); // 0
BinarySearch(data, 57); // 7
BinarySearch(data, 20); // -1
Fix any errors independently. If time remains, explain why the search has time complexity O(log n).

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking.

Binary search reliably finds a target in an unsorted array. True or false? Explain your answer and what low <= high protects.

Circle: Confident / Getting there / Need help

Explain what you can now do independently.

GREEN PEN — Correct and explain

Choose one error. Write the correct method and explain precisely why your first response was wrong.

WE DO — CHECK AND IMPROVE

Source-grounded model answers

CHECK only after completing the guided practice and Exit Ticket. Use GREEN PEN to improve your method and explain any error.

Guided 1 — CHECK

Answer and explanation
target = 13. Return -1 if 13 does not appear.

Guided 2 — CHECK

Answer and explanation
low = 0, high = 7.

Guided 3 — CHECK

Answer and explanation
mid = 3, data[3] = 18.

Guided 4 — CHECK

Answer and explanation
Set high = mid - 1 = 2. Keep indexes 0–2.

Guided 5 — CHECK

Answer and explanation
mid = 1; data[1] = 7.

Guided 6 — CHECK

Answer and explanation
Set low = mid + 1 = 2.

Guided 7 — CHECK

Answer and explanation
mid = 2; data[2] = 12; 12 is not 13.

Guided 8 — CHECK

Answer and explanation
No. The loop ends because 3 > 2. Return -1.

Guided 9 — CHECK

Answer and explanation
CODE:

```
int high = data.Length - 1;
while (low <= high)
{
int mid = low + (high - low) / 2;
if (data[mid] < target)
{
low = mid + 1;
}
}
```

If the midpoint value is smaller than the target, move the lower bound to the right.

Guided 10 — CHECK

Answer and explanation
The smaller midpoint means search right: set low = mid + 1, not high = mid - 1.

WE DO — CHECK AND IMPROVE

Source-grounded model answers · continued

Guided 11 — CHECK

Answer and explanation
B: low=4, high=6. 12 is smaller than 21, so search the right-hand section.